چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۷

تزریق وابستگی (Dependency Injection) — اصول، Hilt، درون Dagger، Koin و KMP



اسماعیل مرادی

@esmaeeil-moradi700

چرا DI روز هفتم است؟

سؤال‌های DI مهندسی را که فقط annotation اضافه می‌کند از مهندسی که **گراف اشیا** را می‌فهمد جدا می‌کند. انتظار مصاحبه‌گر در ۲۰۲۶:

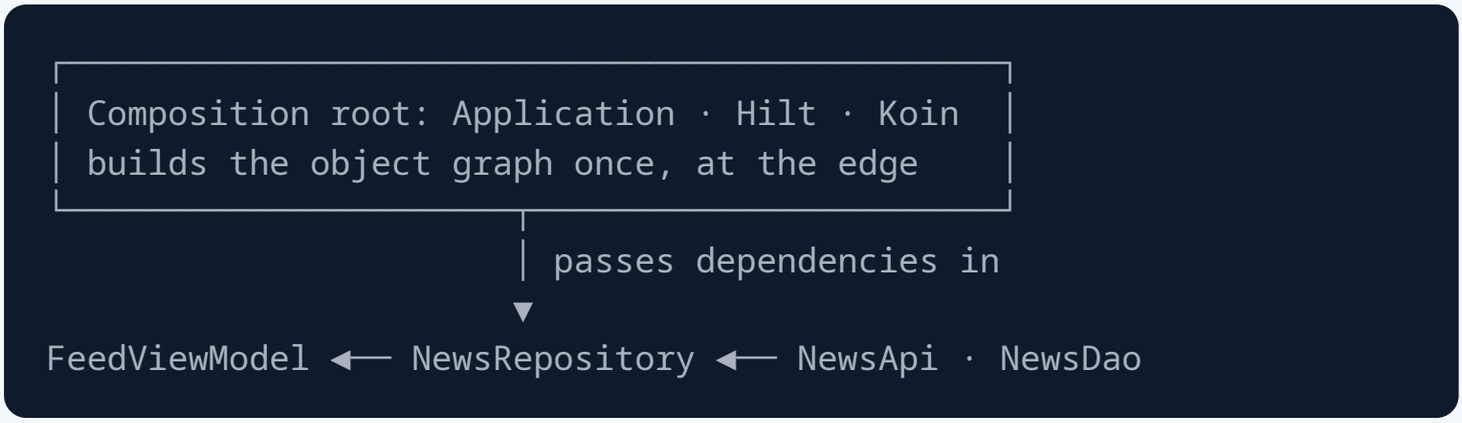
- اول DI به‌عنوان یک **اصل**: تزریق از طریق constructor، وارونگی وابستگی، تست‌پذیری
- **Hilt** به‌عنوان پیش‌فرض اندروید: componentها، scopeها، qualifierها، assisted injection و تست
- اینکه **Dagger چه کدی تولید می‌کند** — و چرا خطاهایش در زمان کامپایل ظاهر می‌شوند
- **Koin** و **DI دستی** به‌عنوان گزینه‌های جدی، مخصوصاً در Kotlin Multiplatform
- kapt در حالت نگه‌داری است — Dagger و Hilt روی **KSP** اجرا می‌شوند

سطح	سؤال‌ها
جونیور	DI چیست، تزریق constructor یا field، راه‌اندازی Hilt
مید	componentها و scopeها، Binds و Provides و qualifierها، assisted injection، تست
سینیور	Dagger چه تولید می‌کند، multibinding و ماژول‌ها، Lazy و Provider و وابستگی چرخه‌ای
آرکیتکت	Hilt یا Koin یا دستی، DI در KMP، DI در مقیاس بزرگ



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: یک composition root



- کلاس‌ها چیزی را که لازم دارند **اعلام می‌کنند** — در constructor — و هیچ‌وقت خودشان نمی‌سازند
- یک جا، یعنی **composition root**، تصمیم می‌گیرد هر کلاس کدام پیاده‌سازی را بگیرد
- Hilt، Koin یا یک container دست‌نویس فقط **همان یک جا را برایتان می‌نویسند**

> **DI اول یک الگوست، بعد یک کتابخانه**
>
> اگر بتوانید گراف‌تان را بدون نام بردن از هیچ فریم‌ورکی توضیح دهید، DI را فهمیده‌اید.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

DI چیست، تزریق constructor یا field، راه‌اندازی Hilt



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. تزریق وابستگی چیست؟

```
// Without DI: hidden, hard-wired dependencies
class NewsRepository {
    private val api = RetrofitNewsApi()
    private val dao = AppDatabase.instance.newsDao()
}

// With DI: dependencies are visible and replaceable
class NewsRepository(
    private val api: NewsApi,
    private val dao: NewsDao,
)

// A unit test just passes fakes
val repo = NewsRepository(FakeNewsApi(), FakeNewsDao())
```

کلاس وابستگی‌هایش را **دریافت می‌کند**، به‌جای اینکه خودش بسازد یا جست‌وجویشان کند. DI همین است — کتابخانه اختیاری است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. چرا مهم است — و چه چیزی نیست

- **وارونگی وابستگی:** به `NewsApi` که یک interface است وابسته باشید، نه به `RetrofitNewsApi` — پیاده‌سازی را بدون دست زدن به کلاس عوض کنید
- **تست‌پذیری:** fakeها را از طریق constructor بدهید — بدون فریم‌ورک در unit test
- **گراف صریح:** constructor هر چیزی را که کلاس لازم دارد فهرست می‌کند

تزریق وابستگی	Service Locator
وابستگی‌ها	در constructor دیده می‌شوند	داخل کلاس پنهان‌اند، مثل `get()`
وابستگی جاافتاده	هنگام سیم‌کشی گراف پیدا می‌شود	در اولین استفاده crash می‌دهد
تست	fakeها را مستقیم پاس می‌دهید	باید یک registry سراسری را تنظیم کنید

تلهٔ مصاحبه

«*DI یعنی همان Dagger یا Hilt است.*» — DI یعنی وابستگی‌ها را از بیرون بدهید. فریم‌ورک فقط سیم‌کشی را خودکار می‌کند؛ یک اپ کوچک می‌تواند آن را دستی انجام دهد (سؤال ۱۱).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. تزریق از طریق constructor، field یا متد؟

نوع	کِی
constructor	کلاس‌های خودتان — پیش‌فرض
field (`@Inject lateinit var`)	وقتی اندروید شیء را می‌سازد: Activity، Fragment، Service، BroadcastReceiver
متد (`@Inject fun`)	به‌ندرت: کاری که باید بعد از ساخت شیء اجرا شود

```
@AndroidEntryPoint
class PlayerService : Service() {
    @Inject lateinit var player: AudioPlayer

    override fun onCreate() {
        super.onCreate() // fields are injected here
        player.prepare()
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. زمان‌بندی و تله‌ها

نقطهٔ ورود	fieldهای تزریق‌شده آماده‌اند بعد از
Activity	`super.onCreate()`
Fragment	`super.onAttach()`
Service	`super.onCreate()`
BroadcastReceiver	`super.onReceive()`

تلهٔ مصاحبه

تزریق field در کلاس‌های خودتان، مثل ViewModel یا repository. وابستگی‌ها پنهان می‌شوند، شیء ممکن است نیمه‌کاره ساخته شود و هر تستی به فریم‌ورک نیاز پیدا می‌کند. از تزریق constructor استفاده کنید.

تلهٔ مصاحبه

پاس دادن repository به پایین درخت composableها. composableها تزریق نمی‌شوند: **route** این ViewModel را می‌گیرد (`hiltViewModel()`) و هر چیزی پایین‌تر فقط state و lambda دریافت می‌کند (روز ۳).

سؤال تکمیلی

«*آیا Fragment می‌تواند تزریق constructor داشته باشد؟*» — بله، با یک `FragmentFactory` (روز ۲). Hilt برای Fragmentها فقط از تزریق field استفاده می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. راه‌اندازی Hilt در یک پروژهٔ مدرن

```
@HiltAndroidApp
class App : Application()

@AndroidEntryPoint
class MainActivity : ComponentActivity()

@HiltViewModel
class FeedViewModel @Inject constructor(
    private val repo: NewsRepository,
    private val handle: SavedStateHandle, // provided by Hilt
) : ViewModel()

@Composable
fun FeedRoute(viewModel: FeedViewModel = hiltViewModel()) {
    /* collect state, render the screen */
}
```

در Gradle: پلاگین `com.google.dagger.hilt.android`، کتابخانهٔ `hilt-android`، و `hilt-compiler` که با **`ksp(…)`** اضافه می‌شود، نه با `kapt(…)`.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. هر annotation چه کار می‌کند

Annotation	کارش
`@HiltAndroidApp`	کامپوننت ریشه یعنی `SingletonComponent` را تولید می‌کند و همهٔ ماژول‌های `@InstallIn` را جمع می‌کند
`@AndroidEntryPoint`	به Activity، Fragment، View، Service یا BroadcastReceiver کامپوننت خودش را می‌دهد و fieldها را تزریق می‌کند
`@HiltViewModel`	ViewModel را ثبت می‌کند؛ `hiltViewModel()` یا `by viewModels()` آن را می‌سازند
`@Inject constructor`	روش ساختن کلاس **خودتان**
`@Module` همراه با `@InstallIn`	روش ساختن تایپ‌هایی که **مال شما نیستند** (interfaceها، Retrofit، Room)
`@EntryPoint`	دسترسی به گراف جایی که Hilt نمی‌تواند تزریق کند، مثلاً یک `ContentProvider`

تلهٔ مصاحبه

یک Fragment با `@AndroidEntryPoint` داخل Activityی که annotation ندارد، در runtime crash می‌دهد. Hilt لازم دارد Activity میزبان هم entry point باشد.



اسماعیل مرادی
@esmaeeil-moradi700

میدcomponentها و scopeها، Binds و Provides و qualifierها، assisted injection، تست



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. componentها و scopeها در Hilt

```
SingletonComponent              @Singleton
├─ ActivityRetainedComponent    @ActivityRetainedScoped
│  ├─ ViewModelComponent        @ViewModelScoped
│  └─ ActivityComponent         @ActivityScoped
│     ├─ FragmentComponent      @FragmentScoped
│     └─ ViewComponent          @ViewScoped
└─ ServiceComponent             @ServiceScoped
```

- هر component فرزند همهٔ bindingهای والدهایش را می‌بیند — هرگز برعکس. (`ViewWithFragmentComponent` زیر `FragmentComponent` قرار دارد.)
- binding دارای scope یعنی **یک نمونه برای هر نمونه از component**. بدون scope یعنی برای هر تزریق یک نمونهٔ جدید.
- `ViewModelComponent` به‌طور خودکار `SavedStateHandle` را فراهم می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. انتخاب scope

Scope	یک نمونه برای هر...	کاربرد رایج
`@Singleton`	process اپ	`OkHttpClient`، دیتابیس، cacheهای مشترک
`@ActivityRetainedScoped`	Activity، با عبور از چرخش صفحه	stateی که بین ViewModelهای یک Activity مشترک است
`@ViewModelScoped`	ViewModel	یک شیء مشترک بین همهٔ وابستگی‌های یک ViewModel
`@ActivityScoped` و `@FragmentScoped`	نمونهٔ Activity یا Fragment	به‌ندرت: helperهایی که به همان نمونهٔ UI گره خورده‌اند

تلهٔ مصاحبه

نسخهٔ پارسال می‌گفت ViewModelها در `ActivityRetainedComponent` زندگی می‌کنند و ViewModel یک Fragment در `FragmentComponent`. — هر `@HiltViewModel` یک `ViewModelComponent` **مخصوص خودش** دارد؛ طول عمرش را `ViewModelStoreOwner` تعیین می‌کند (روز ۶).

تلهٔ مصاحبه

«*برای کارایی به همه چیز scope بدهید.*» — scope هزینه دارد: کد تولیدی بیشتر، یک قفل double-checked (`DoubleCheck`)، و شیئی که تا پایان عمر component در حافظه می‌ماند. توصیهٔ گوگل: scope را کم و سنجیده استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. `@Binds` ، `@Provides` و qualifierها

```
@Module
@InstallIn(SingletonComponent::class)
abstract class DataModule {
    @Binds
    abstract fun newsRepo(
        impl: OfflineFirstNewsRepository,
    ): NewsRepository
}

@Module
@InstallIn(SingletonComponent::class)
object DispatchersModule {
    @Provides @IoDispatcher
    fun io(): CoroutineDispatcher = Dispatchers.IO
}

@Qualifier @Retention(AnnotationRetention.BINARY)
annotation class IoDispatcher
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. کدام، و چرا

	`@Binds`	`@Provides`
شکل تعریف	تابع abstract با یک پارامتر	تابع با بدنه
کاربرد	وصل کردن interface به پیاده‌سازی خودتان	تایپ‌هایی که مال شما نیستند: Retrofit، Room، OkHttp، dispatcherها
کد تولیدی	کمتر: Dagger مستقیم factory پیاده‌سازی را صدا می‌زند	یک factory برای هر متد

تلهٔ مصاحبه

دو binding از یک تایپ بدون qualifier یعنی خطای کامپایل `[Dagger/DuplicateBindings]`. به‌جای `@Named("io")` از annotationهای `@Qualifier` سفارشی استفاده کنید: بررسی تایپ دارند، در refactor امن‌اند و خودشان مستندند.

سؤال تکمیلی

«*پیاده‌سازی متفاوت برای debug و release؟*» — در **زمان build** تصمیم بگیرید: یک ماژول برای هر source set، یعنی `src/debug` و `src/release`. فقط تصمیم‌های واقعاً runtime، مثل feature flagها، به انتخاب در runtime نیاز دارند (سؤال ۱۳).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. assisted injection: پارامترهای runtime

```
class ChatSession @AssistedInject constructor(
    private val api: ChatApi,
    @Assisted private val roomId: String,
) {
    @AssistedFactory
    interface Factory {
        fun create(roomId: String): ChatSession
    }
}

// Inject the factory anywhere
class ChatManager @Inject constructor(
    private val sessions: ChatSession.Factory,
) {
    fun open(roomId: String) = sessions.create(roomId)
}
```

Dagger مقدار `api` را می‌دهد و صدازننده مقدار `roomId` را. دیگر لازم نیست کلاس factory را دستی بنویسید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. ViewModelهای assisted

```kotlin
@HiltViewModel(assistedFactory = ItemViewModel.Factory::class)
class ItemViewModel @AssistedInject constructor(
    @Assisted val item: ItemPreview,
    private val repo: ItemRepository,
) : ViewModel() {
    @AssistedFactory
    interface Factory {
        fun create(item: ItemPreview): ItemViewModel
    }
}

// Compose
val vm = hiltViewModel<ItemViewModel, ItemViewModel.Factory> {
    it.create(preview)
}
```

تلهٔ مصاحبه

استفاده از assisted injection برای آرگومان‌های navigation. آن‌ها همین حالا در `SavedStateHandle` هستند و از process death عبور می‌کنند. assisted را برای مقدارهایی بگذارید که در saved state نیستند.

تلهٔ مصاحبه

تایپ‌های assisted **نمی‌توانند scope داشته باشند** — هر `create()` یک نمونهٔ جدید است. دو پارامتر `@Assisted` از یک تایپ به شناسه نیاز دارند: `@Assisted("from")`.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. تست با Hilt

```
@HiltAndroidTest
@UninstallModules(PaymentModule::class)
class CheckoutFlowTest {
    @get:Rule(order = 0) val hiltRule = HiltAndroidRule(this)

    @get:Rule(order = 1)
    val composeRule = createAndroidComposeRule<MainActivity>()

    @BindValue @JvmField
    val gateway: PaymentGateway = FakePaymentGateway()

    @Test fun paysWithTheFakeGateway() {
        composeRule.onNodeWithText("Pay").performClick()
        composeRule.onNodeWithText("Paid").assertIsDisplayed()
    }
}
```

`@BindValue` یک binding را فقط برای همین کلاس تست جایگزین می‌کند؛ ماژول اصلی باید uninstall شود، وگرنه binding تکراری می‌گیرید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. جایگزینی ماژول در همهٔ تست‌ها

```
@Module
@TestInstallIn(
    components = [SingletonComponent::class],
    replaces = [PaymentModule::class],
)
abstract class FakePaymentModule {
    @Binds
    abstract fun gateway(
        fake: FakePaymentGateway,
    ): PaymentGateway
}
```

- تست instrumented یک `AndroidJUnitRunner` سفارشی می‌خواهد که `HiltTestApplication` برگرداند؛ در Robolectric هم `@Config(application = HiltTestApplication::class)` کافی است.

تلهٔ مصاحبه

نسخهٔ پارسال `testInstrumentationRunner` را «برای unit test» برابر `"androidx.hilt.work.testing.HiltTestApplication_Application"` گذاشته بود. چنین کلاسی وجود ندارد — و این تنظیم فقط روی تست‌های **instrumented** اثر دارد.

تلهٔ مصاحبه

«*unit test یک ViewModel به Hilt نیاز دارد.*» — نه. تزریق constructor یعنی `FeedViewModel(FakeNewsRepository())`. تست‌های Hilt برای تست‌های integration و UI هستند.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

Dagger چه تولید می‌کند، multibinding و ماژول‌ها، Lazy و Provider و وابستگی چرخه‌ای



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. Dagger چه کدی تولید می‌کند

You write	Dagger generates (Java)
@Inject constructor	UserRepo_Factory
@Provides fun api()	NetworkModule_ApiFactory
@Component / Hilt root	the component class
@Singleton binding	DoubleCheck memoization

```
// Simplified shape of the generated code
class UserRepo_Factory(
    private val api: Provider<Api>,
    private val dao: Provider<UserDao>,
) : Factory<UserRepo> {
    override fun get() = UserRepo(api.get(), dao.get())
}

// Inside the component, for a @Singleton binding:
val userRepo: Provider<UserRepo> = DoubleCheck.provider(
    UserRepo_Factory(apiProvider, daoProvider),
)
```

در runtime هیچ reflectionی نیست — فقط صدا زدن constructorها.
کل گراف **در زمان کامپایل** اعتبارسنجی می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. Hilt چه چیزی اضافه می‌کند

- `@HiltAndroidApp` یک کلاس پایهٔ `Hilt_App` تولید می‌کند؛ پلاگین Gradle مربوط به Hilt کلاس والد شما را در bytecode عوض می‌کند، برای همین همچنان `Application() :` می‌نویسید.
- وقتی `app:` کامپایل می‌شود، Hilt **همهٔ ماژول‌های `@InstallIn`** موجود در گراف وابستگی‌اش را جمع می‌کند و کل سلسله‌مراتب componentها را تولید می‌کند.
- مشکلات گراف خطای کامپایل‌اند: `[Dagger/MissingBinding]`، `[Dagger/DependencyCycle]` و `[Dagger/DuplicateBindings]`.

تلهٔ مصاحبه

«*Dagger کند است چون از reflection استفاده می‌کند.*» — Dagger 2 کد ساده تولید می‌کند. هزینه‌اش در **زمان build** است (پردازش annotationها)، نه در runtime.

تلهٔ مصاحبه

«*اشیای `@Singleton` هنگام شروع اپ ساخته می‌شوند.*» — Dagger آن‌ها را به‌صورت lazy و در اولین درخواست می‌سازد. اما هر چیزی که در fieldهای `Application` تزریق شود، در `Application.onCreate()` ساخته می‌شود — یعنی هزینهٔ startup (روز ۱).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. multibinding: ماژول‌ها سهم می‌دهند، اپ جمع می‌کند

```
interface AppInitializer { fun init(app: Application) }

// :feature:analytics:impl
@Module @InstallIn(SingletonComponent::class)
abstract class AnalyticsInitModule {
    @Binds @IntoSet
    abstract fun bind(
        impl: AnalyticsInitializer,
    ): AppInitializer
}

@HiltAndroidApp
class App : Application() {
    @Inject lateinit var initializers:
        Set<@JvmSuppressWildcards AppInitializer>

    override fun onCreate() {
        super.onCreate()
        initializers.forEach { it.init(this) }
    }
}
```

ابزارها: `@IntoSet`، `@IntoMap` همراه با `@StringKey` یا `@ClassKey`، و `@Multibinds` برای مجموعه‌ای که ممکن است خالی باشد. روز ۲ با همین ایده گراف‌های navigation را اضافه کرد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. DI بین ماژول‌های Gradle

- `@HiltAndroidApp` فقط در `:app` است. Hilt از همهٔ ماژول‌های `@InstallIn` که `:app` به آن‌ها وابسته است **یک** سلسله‌مراتب component می‌سازد.
- interfaceها در `:feature:x:api` یا `:core:domain`؛ پیاده‌سازی‌ها و `@Binds` آن‌ها در `:impl`. فیچرها فقط به `:api` وابسته‌اند.
- پیاده‌سازی‌ها را `internal` کنید — کپسوله‌سازی از دیدپذیری Gradle می‌آید، نه از Hilt.

تلهٔ مصاحبه

نسخهٔ پارسال می‌گفت هر ماژول گراف جدا و محصور خودش را دارد. — در Hilt فقط یک گراف هست: bindingی که در `SingletonComponent` است، برای هر ماژولی که تایپش را ببیند در دسترس است.

تلهٔ مصاحبه

نسخهٔ پارسال می‌گفت `@AndroidEntryPoint` در dynamic feature module کار می‌کند و Hilt بعد از دانلود، ماژول‌هایش را اضافه می‌کند. — نه. اپ پایه در زمان کامپایل از dynamic feature خبر ندارد، پس Hilt نمی‌تواند پردازشش کند. در آن فیچر از Dagger ساده استفاده کنید، با وابستگی component به یک `@EntryPoint` از اپ.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. `Lazy` ، `Provider` و وابستگی چرخه‌ای

```
class ReportExporter @Inject constructor(
    // dagger.Lazy: created on first get(), then cached
    private val pdf: Lazy<PdfRenderer>,
    // new instance per get() — unless the binding is scoped
    private val upload: Provider<UploadRequest>,
) {
    fun export(report: Report) {
        val file = pdf.get().render(report)
        upload.get().send(file)
    }
}

class A @Inject constructor(private val b: B)
class B @Inject constructor(private val a: A)
// compile error: [Dagger/DependencyCycle]
```

`Lazy` اشیای سنگین و کم‌استفاده را از مسیر startup بیرون نگه می‌دارد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. جاهایی که اشتباه می‌کنند

تلهٔ مصاحبه

نسخهٔ پارسال می‌گفت: «`Provider.get()` همیشه نمونهٔ جدید برمی‌گرداند.» — فقط برای bindingهای بدون scope. برای binding دارای scope، هر `get()` همان نمونه را برمی‌گرداند.

تلهٔ مصاحبه

نسخهٔ پارسال می‌گفت: «*وابستگی چرخه‌ای یا خطای کامپایل است یا در runtime به* `StackOverflowError` *می‌رسد.*» — Dagger هر چرخه را در زمان کامپایل گزارش می‌کند. فقط وقتی کامپایل می‌شود که یکی از یال‌ها `Lazy` یا `Provider` باشد — و اگر داخل constructor متد `get()` را صدا بزنید، باز هم overflow می‌شود.

تلهٔ مصاحبه

import کردن `kotlin.Lazy` به‌جای `dagger.Lazy` یعنی خطای `[Dagger/MissingBinding]`. `Lazy` در کاتلین تایپ Dagger نیست.

چرخه را با طراحی حل کنید

منطق مشترک را در کلاس سوم بیرون بکشید، یا یکی از یال‌ها را با یک interface، callback یا `Flow` وارونه کنید. `Lazy` و `Provider` فقط بوی بد را پنهان می‌کنند.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

Hilt یا Koin یا دستی، DI در KMP، DI در مقیاس بزرگ



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. Hilt، Koin یا DI دستی؟

	Hilt و Dagger	Koin	دستی
بررسی گراف	زمان کامپایل	در runtime — یا با یک تست `verify()`	توسط کامپایلر؛ کد معمولی است
هزینهٔ build	پردازش annotation با KSP	ندارد	ندارد
runtime	کد تولیدی، بدون reflection	جست‌وجو در registry هنگام resolve	صدا زدن مستقیم
چندسکویی	نه — فقط JVM و اندروید	بله	بله
بهترین کاربرد	اپ‌های بزرگ اندروید، تیم‌های زیاد	KMP، اپ‌های کوچک تا متوسط	اپ‌های کوچک، کتابخانه‌ها، SDKها

برای KMP گزینه‌های compile-time هم هست: **kotlin-inject** که روی KSP کار می‌کند و **Metro** که یک compiler plugin است.

تلهٔ مصاحبه

«*Koin فقط یک service locator است.*» — تعریف ماژول با پارامترهای constructor همان DI است. صدا زدن `get()` یا `by inject()` **داخل** کلاس‌های بیزینسی، service locator است: وابستگی پنهان و خطا در runtime.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. همان گراف با Koin و به‌صورت دستی

```
// Koin
val appModule = module {
    singleOf(::OfflineFirstNewsRepository) {
        bind<NewsRepository>()
    }
    viewModelOf(::FeedViewModel)
}

// Manual DI: a container is just a class
class AppContainer(context: Context) {
    private val db by lazy { AppDatabase.build(context) }
    private val api by lazy { NewsApi(HttpClient()) }
    val newsRepository: NewsRepository by lazy {
        OfflineFirstNewsRepository(db.newsDao(), api)
    }
}
```

- در Koin یک unit test با `verify()` از koin-test اضافه کنید تا تعریف جاافتاده در CI شکست بخورد، نه روی گوشی کاربر.
- DI دستی از طریق `viewModelFactory` و `APPLICATION_KEY` به ViewModelها وصل می‌شود (روز ۶).
- SDK می‌نویسید؟ فریم‌ورک DI را به کاربرانتان تحمیل نکنید — constructor یا builder در اختیارشان بگذارید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. DI در Kotlin Multiplatform

```
// commonMain
expect val platformModule: Module

val sharedModule = module {
    singleOf(::OfflineFirstNewsRepository) {
        bind<NewsRepository>()
    }
    viewModelOf(::FeedViewModel)
}

fun initKoin(config: KoinAppDeclaration = {}) = startKoin {
    config()
    modules(sharedModule, platformModule)
}

// androidMain
actual val platformModule = module {
    single<DbFactory> { AndroidDbFactory(androidContext()) }
}
```

اندروید `initKoin { androidContext(this@App) }` را صدا می‌زند. در `iosMain` کلاس `IosDbFactory()` bind می‌شود و Swift این تابع را با نام `doInitKoin()` می‌بیند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. قواعد کد مشترک

- interfaceها در `commonMain` هستند و پیاده‌سازی‌های هر پلتفرم **تزریق می‌شوند.** `expect` و `actual` را برای factory کوچک پلتفرم در لبهٔ سیستم نگه دارید.
- گزینه‌ها: Koin (runtime و رایج‌ترین)، kotlin-inject یا Metro (compile-time)، یا containerهای دستی.
- Hilt می‌تواند در ماژول اپ اندروید بماند و کلاس‌های مشترک را با `@Provides` فراهم کند — فقط نمی‌تواند در `commonMain` باشد.

تلهٔ مصاحبه

«*در کد مشترک از Hilt استفاده کنید.*» — Dagger و Hilt فقط برای JVM و اندروید هستند و روی iOS اجرا نمی‌شوند.

تلهٔ مصاحبه

یک `expect class` برای هر وابستگی پلتفرمی. این کار برای هر پلتفرم یک پیاده‌سازی را سیم‌کشی ثابت می‌کند و جایی برای fake در تست‌های مشترک نمی‌گذارد. interface به‌علاوهٔ DI را ترجیح دهید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. عوض کردن پیاده‌سازی در runtime

```
@Module @InstallIn(SingletonComponent::class)
abstract class CheckoutModule {
    @Binds @IntoMap @StringKey("classic")
    abstract fun classic(impl: ClassicCheckout): Checkout

    @Binds @IntoMap @StringKey("express")
    abstract fun express(impl: ExpressCheckout): Checkout
}

class CheckoutSelector @Inject constructor(
    private val variants:
        Map<String, @JvmSuppressWildcards Provider<Checkout>>,
    private val flags: FeatureFlags,
) {
    fun current(): Checkout =
        variants.getValue(flags.checkoutVariant()).get()
}
```

گراف ثابت می‌ماند؛ فقط **انتخاب** در runtime انجام می‌شود. با `Provider` فقط پیاده‌سازی انتخاب‌شده ساخته می‌شود.

تلهٔ مصاحبه

«سیستم DI با hot-swap» در نسخهٔ پارسال، یعنی یک switcher سفارشی که از طریق entry point در دسترس بود، زیادی پیچیده است. گراف Dagger در زمان کامپایل ثابت است — گزینه‌ها را bind کنید، بعد انتخاب کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. سریع و سالم نگه داشتن یک گراف بزرگ

دغدغه	روش
زمان build	اجرای Dagger و Hilt روی KSP؛ ترجیح `@Binds` به `@Provides`؛ ماژول‌های کوچک و پایدار
startup	هیچ چیز سنگینی در fieldهای `Application` نباشد؛ `Lazy` برای اشیای سنگین و کم‌استفاده
حافظه	scope فقط برای state مشترک یا اشیای گران
درستی	خطاهای گراف در زمان کامپایل؛ گزینهٔ `dagger.fullBindingGraphValidation` bindingهای بلااستفاده را هم چک می‌کند؛ `verify()` در Koin داخل CI
مرزها	ماژول‌های `api` و `impl`، پیاده‌سازی‌های `internal`

جواب سینیور در یک جمله

«constructorها اعلام می‌کنند چه لازم دارند، یک composition root تصمیم می‌گیرد چه بگیرند — و فریم‌ورک، چه Hilt چه Koin چه هیچ‌کدام، یک trade-off است که می‌توانم از آن دفاع کنم.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۷ — خلاصهٔ یک‌صفحه‌ای

- DI یعنی وابستگی‌ها را دریافت کنید، نه اینکه بسازید.
- تزریق constructor پیش‌فرض است؛ field فقط جایی که اندروید شیء را می‌سازد.
- Hilt را با KSP کامپایل کنید، نه kapt.
- هر ViewModel یک `ViewModelComponent` مخصوص خودش دارد؛ scope کم.
- برای interfaceهای خودتان `@Binds`، برای تایپ‌های دیگران `@Provides`، و به‌جای `@Named` یک qualifier سفارشی.
- assisted injection برای مقدارهای runtime؛ آرگومان navigation در `SavedStateHandle` است.
- unit test با constructor و fake؛ تست UI با `@BindValue` و `@TestInstallIn` است.
- Dagger گراف را در زمان کامپایل چک می‌کند؛ `@Singleton` به‌صورت lazy ساخته می‌شود.
- فقط بدون scope، `Provider` هر بار نمونهٔ جدید می‌دهد. چرخه را با طراحی حل کنید.
- در KMP از Koin، kotlin-inject، Metro یا DI دستی استفاده کنید؛ Hilt سمت اندروید می‌ماند.

روز ۸ در راه است: مبانی Jetpack Compose. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700